

C++ STL

```
#include <bits/stdc++.h>
using namespace std;

//
typedef long long ll;
typedef long double ld;
typedef pair<int, int> PII;
typedef pair<ll, ll> PLL;

//
const int INF = 1e9;
const ll LLINF = 1e18;
const int MOD = 1e9 + 7;
const double PI = acos(-1.0);
```

vector

1.

```
vector<int> v;           // int
vector<int> v(10);       // 10      0
vector<int> v(10, 5);    // 10      5
vector<int> v = {1,2,3}; //
```

2.

v.push_back(x)	x	O(1)

<code>v.emplace_back(x)</code>	x push_back	O(1)
<code>v.pop_back()</code>		O(1)
<code>v.size()</code>		O(1)
<code>v.empty()</code>		O(1)
<code>v.clear()</code>		O(n)
<code>v.resize(n)</code>	n	O(n)
<code>v.resize(n, val)</code>	n val	O(n)
<code>v.reserve(n)</code>	n size	O(n)
<code>v.erase(it)</code>	it	O(n)
<code>v.erase(first, last)</code>	[first, last) last	O(n)

3.

```
// 1.          Dijkstra/BFS/DFS
vector<vector<PII>> ver(n + 1);
ver[x].eb(y, w); //    x→y    w

// 2.
for (int i = 0; i < v.size(); i++) {
    cout << v[i];
}
//    for C++11+
for (auto x : v) {
    cout << x;
}

// 3.
sort(all(v));
v.erase(unique(all(v)), v.end());

// 4.
for (auto it = v.begin(); it != v.end(); ) {
    if (*it == target) {
        it = v.erase(it); // erase
    } else {
        it++;
    }
}
```

4.

- `i < v.size()`
- `reserve()` `push_back / emplace_back`
- `vector` `== / !=`
- `erase`

pair

1.

```
PII p;  
pair<int, string> p = {1, "abc"};
```

2.

```
p.fi; // p.first  
p.se; // p.second
```

3.

```
// 1.  
PII edge = {y, w};  
  
// 2.  
priority_queue<PII> q;  
  
// 3. C++17  
for (auto [y, w] : ver[x]) {  
    // y w  
}  
//  
for (auto &e : ver[x]) {  
    int y = e.fi;  
    int w = e.se;  
}
```

1. string

```
string s = "abc";
s.size(); //
s += "def"; //
s[0]; //
s.substr(pos, len); // pos len
s.find("abc");// string::npos
s.erase(pos); // pos
s.erase(pos, len); // pos len
s.erase(it); // it
s.erase(first, last); // [first, last) last
```

```
// 'a'
for (auto it = s.begin(); it != s.end(); ) {
    if (*it == 'a') {
        it = s.erase(it);
    } else {
        it++;
    }
}
```

- vector erase

2. deque

O(1)

```

deque<int> dq;           //
deque<int> dq(10);       // 10           0
deque<int> dq(10, 5);    // 10           5
deque<int> dq = {1,2,3}; //

```

<code>dq.push_front(x)</code>	x	O(1)
<code>dq.push_back(x)</code>	x	O(1)
<code>dq.emplace_front(x)</code>	x	O(1)
<code>dq.emplace_back(x)</code>	x	O(1)
<code>dq.pop_front()</code>		O(1)
<code>dq.pop_back()</code>		O(1)
<code>dq.front()</code>		O(1)
<code>dq.back()</code>		O(1)
<code>dq[i]</code>	i	O(1)
<code>dq.size()</code>		O(1)
<code>dq.empty()</code>		O(1)
<code>dq.clear()</code>		O(n)
<code>dq.erase(it)</code>	it	O(n)
<code>dq.erase(first, last)</code>	[first, last) last	O(n)

① BFS queue

```
deque<int> dq;
vector<int> dis(n + 1, -1);

dq.push_back(1);
dis[1] = 0;

while (!dq.empty()) {
    int u = dq.front();
    dq.pop_front();

    for (auto [v, w] : ver[u]) {
        if (dis[v] == -1) {
            dis[v] = dis[u] + 1;
            dq.push_back(v);
        }
    }
}
```

② 0-1 BFS

0 1

```
deque<PII> dq;
vector<int> dis(n + 1, INF);

dq.push_front({0, s});
dis[s] = 0;

while (!dq.empty()) {
    auto [d, u] = dq.front();
    dq.pop_front();

    if (d > dis[u]) continue;

    for (auto [v, w] : ver[u]) {
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            if (w == 0) {
                dq.push_front({dis[v], v}); // 0
            } else {
                dq.push_back({dis[v], v}); // 1
            }
        }
    }
}
```

•

erase

vector

3. list

$O(1)$


```

list<int> l;
l.push_back(x); //
l.push_front(x); //
l.pop_back();    //
l.pop_front();   //
l.front();       //
l.back();        //
l.size();        //
l.empty();       //
l.clear();       //
l.erase(it);     //
l.erase(first, last); //    [first, last)    last

```

- []
-

1. priority_queue /

```

//
priority_queue<int> q;
//      Dijkstra
priority_queue<int, vector<int>, greater<int>> q;

// pair      Dijkstra
priority_queue<PII, vector<PII>, greater<PII>> q;

```

q.push(x)	x

q.emplace(x)	x
q.top()	
q.pop()	
q.empty()	
q.size()	

 **priority_queue** **erase**

2. queue BFS

```
queue<int> q;
q.push(x);    //
q.front();    //
q.back();     //
q.pop();      //
q.empty();    //
q.size();     //
```

queue **erase**

deque

3. stack

```
stack<int> s;
s.push(x);    //
s.top();      //
s.pop();      //
s.empty();    //
s.size();     //
```

****** **stack** **erase** ******

- pop()
-
- vector deque

1. set

```
set<int> st;
st.insert(x);    // x
st.count(x);     // x      0 1
st.find(x);      // x      st.end()
st.size();       //
st.empty();      //
st.clear();      //
st.erase(val);   // val
st.erase(it);    // it
st.erase(first, last); // [first, last) last
```

```
for (auto it = st.begin(); it != st.end(); ) {
    if (*it == target) {
        it = st.erase(it);
    } else {
        it++;
    }
}
```

-
-

2. map

```
map<int, int> mp;
mp[key] = value; //      /
mp.count(key);   //      key      0 1
mp.find(key);    //      key      mp.end()
mp.size();       //
mp.empty();      //
mp.clear();      //
mp.erase(key);   //      key
mp.erase(it);    //      it
mp.erase(first, last); // [first, last)      last
```

```
for (auto it = mp.begin(); it != mp.end(); ) {
    if (it->se == target) { //      target
        it = mp.erase(it);
    } else {
        it++;
    }
}
```

-
- set

3. multiset / multimap

```
set/map      erase(val)      val
val
```

```

auto it = st.find(val);
if (it != st.end()) {
    st.erase(it);
}

```

1. unordered_set / unordered_map

```

unordered_set<int> st;
st.insert(x);      //      x
st.count(x);       //      x
st.find(x);        //      x
st.size();         //
st.empty();        //
st.clear();        //
st.erase(val);     //      val
st.erase(it);      //      it
st.erase(first, last); // [first, last)      last

```

- $O(1)$
-

2. unordered_multiset / unordered_multimap

unordered_set/unordered_map erase(val)

1. sort $O(n \log n)$

```
//
sort(all(v));
//
sort(all(v), greater<int>());
//
bool cmp(const int &a, const int &b) {
    return a > b; //
}
sort(all(v), cmp);
```

2.

```
max(a, b);    //
min(a, b);    //
swap(a, b);   //
*max_element(all(v)); //
*min_element(all(v)); //
```

3. $O(\log n)$

```
//          >= x
auto it = lower_bound(all(v), x);
int pos = it - v.begin();

//          > x
auto it = upper_bound(all(v), x);
```

4.

```
//
sort(all(v));
v.erase(unique(all(v)), v.end());
```

erase

	erase		
vector/deque/string			O(n)
list			O(1)
set/map/multiset/multimap	C++11+		O(log n)
unordered_*	C++11+		O(1)

erase

- 1. erase
- 2. erase
- 3. erase erase find()
- 4. multiset/multimap find() erase

1.

```
vector<vector<PII>> ver(n + 1);
for (int i = 0; i < m; i++) {
    int x, y, w;
    cin >> x >> y >> w;
    ver[x].eb(y, w);
    //
    // ver[y].eb(x, w);
}
```

2. /

```
vector<int> cnt(26, 0);
cnt[c - 'a']++; // c
```

3. Dijkstra

```
priority_queue<PII, vector<PII>, greater<PII>> q;
q.eb(0, s); // ( )
```

- 1. int double 1.0 * a / b
- 2. int 2e9 long long INF int LLINF long long
- 3. **priority_queue** Dijkstra greater<PII>
- 4. **vector** resize() push_back()
- 5. **C++17** auto [y, w] auto &e